

PR01 PROGRAMMATION ORIENTÉE OBJET

Définition 1 : Programmation Orientée Objet - POO

La **Programmation Orientée Objet** ou **POO** est un paradigme de programmation informatique. Il consiste en la définition et l'interaction de briques logicielles appelées **objets**. Chaque objet est une **instance** de **classes** qui définissent le comportement et les propriétés des objets.

Exemple : Exemple de classes et d'objets

- **Pokemon** est une classe ; c'est un concept. Pour le moment aucun pokemon n'existe mais juste l'idée ;
- **pikachu** est un objet. C'est un pokemon particulier. On dit que c'est une instance de classe ;
- une classe possède un nom et des caractéristiques (on parle d'**attributs**) ;
- une classe possède des fonctionnalités (on parle de **méthodes**)



Classe = Type d'objet
Objet = Instance de classe

Nom de la classe: Pokemon

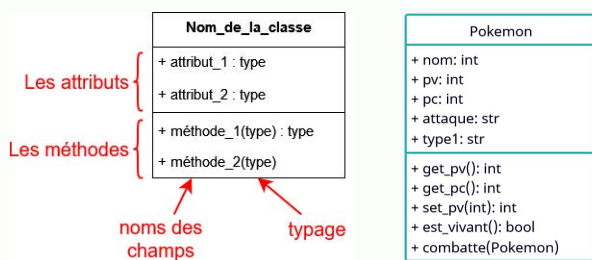
Attributs:

- nom (str)
- points de vie: PV (int)
- points de combat: PC (int)
- attaque (str)
- type (str)

Méthodes:

- connaître les PV du pokemon
- connaître les PC du pokemon
- modifier les PV du pokemon
- savoir si le pokemon est toujours vivant

Propriété 1 : Diagramme de classe UML



- **Pokemon** est le nom de la classe
- **Pikachu** est une instance de la classe **Pokemon**
- **pikachu** est la valeur de l'attribut **nom** de l'instance **Pikachu**
- **60** est la valeur de l'attribut **pv** de l'instance **Pikachu**

Propriété 2

En python tout est objet En python, toutes les variables sont déjà des objets. Pour vous en convaincre : `dir(list)`

Dans la liste des méthodes associées à la classe `list`, on retrouve les méthodes connues (`append`, `pop`...) mais également des méthodes spéciales (ou **méthodes magiques**) qui sont entourées de double underscores (`__init__`, `__len__`, `__add__`, etc.). Ces méthodes permettent de définir le comportement des objets de la classe.

I. Construire une classe

Définition 1 : Construire une classe

Une classe est au minimum constituée d'un constructeur de classe.

</> Code Python

```
1 class Point:                                # Définition du nom de la classe
2     def __init__(self, abs, ord):           # Constructeur de classe
3         self.x = abs                        # Attribut x
4         self.y = ord                       # Attribut y
```

- `class Point` : c'est la porte d'entrée de la classe ;
- `def __init__(self, abs, ord)` : c'est la méthode qui est appelée au moment de la création d'une instance de la classe `Point` ;
- `abs` et `ord` sont des paramètres à entrer au moment de créer l'instance ;
- `self` : c'est l'objet en lui-même. Pour le moment, il n'a pas de nom puisqu'on crée la classe. Ensuite `self` sera remplacé par le nom de l'instance de la classe ;
- `self.x` : on crée un attribut de classe `x` qui prend l'abscisse pour valeur ;
- `self.y` : on crée un attribut de classe `y` qui prend l'ordonnée pour valeur.

Définition 2 : Se servir d'une classe

Pour se servir d'une classe, il suffit maintenant de créer un objet (une instance) de la classe `Point`

</> Code Python

```
1 A = Point(1, 7)                            # A est une instance de la classe Point
2 B = Point(0, 4)                            # B est également une instance de la classe Point
```

Au moment de la création de l'instance `A`, le constructeur est exécuté. Dans toute la suite, `self` sera donc remplacé par `A`.

</> Code Python

```
1 print(A.x)                                # On récupère l'attribut x de l'instance A. Affichage 1
2 print(A.y)                                # On récupère l'attribut y de l'instance A. Affichage 7
3
4 dx = B.x - A.x                            # On calcule la différence d'abscisse entre les points A et B
5 dy = B.y - A.y                            # On calcule la différence d'ordonnée entre les points A et B
6
7 coeff_directeur = dy/dx                   # On calcule le coefficient directeur de la droite (AB)
```

Définition 3 : Accesseurs et mutateurs

En général on s'interdit d'accéder aux attributs de l'instance de la classe directement. A la place on crée des méthodes permettant d'accéder à ces attributs :

- **Accesseurs** (getters) : permet de récupérer la valeur de l'attribut ;
- **Mutateurs** (setters) : permet de modifier la valeur de l'attribut.

</> Code Python

```
1 class Point:                                # Définition du nom de la classe
2     def __init__(self, abs, ord):           # Constructeur de classe
3         self.x = abs                        # Attribut x
4         self.y = ord                       # Attribut y
5     def get_coord(self):
6         ''' Renvoie les coordonnées du point sous forme de tuple '''
7         return (self.x, self.y)
8     def set_coord(self, coord:tuple):
9         ''' Modifie les coordonnées du point '''
10        self.x, self.y = coord
```

Affichage

Python permet d'afficher de deux manières : soit via la fonctionnalité `print` soit directement dans la console. Derrière ces fonctionnalités, il se cache deux méthodes :

- Pour la fonction `print`, c'est la méthode `__str__`
- Pour l'affichage dans la console c'est la méthode `__repr__`

Par défaut ces méthodes sont créées lors de la création de la classe `Point` mais ne donnent rien d'intéressant :

</> Code Python

```
1 >>> print(A)
2 <__main__.Point object at 0x00000233824A4D08>
3 # A est un objet de type Point situé à l'adresse mémoire 0x00000233824A4D08
```

On va donc surcharger (modifier) ces deux méthodes

</> Code Python

```
1 def __str__(self):
2     ''' Surcharge la fonction print. Affichage 'Coordonnées = (abs;ord)' '''
3     return f'Coordonnées = ({self.x};{self.y})'
4 def __repr__(self)
5     ''' Surcharge __repr__. Affichage (abs;ord)' '''
6     return f'({self.x};{self.y})'
```

Définition 4 : D'autres méthodes intéressantes

Dans le même ordre d'idée, il est possible de redéfinir des méthodes comme :

- `__eq__(self, objet)` qui renvoie `True` si `self == objet` ;
- `__gt__(self, objet)` qui renvoie `True` si `self > objet` ;
- `__lt__(self, objet)` qui renvoie `True` si `self < objet` ;
- `__add__(self, objet)` qui renvoie `self + objet` ;
- `__mul__(self, objet)` qui renvoie `self * objet` ;

Propriété 1 : Utilisation des méthodes

</> Code Python

```
1 >>> A = Point(1, 6)
2 >>> B = Point(2, 7)
3 >>> C = Point(1, 6)
4 >>> B.set_coord((4, 8))
5 >>> B.get_coord()
6 (4, 8)
7 >>> print(B)
8 Coordonnées = (4;8)
9 >>> B
10 (4;8)
11 >>> A == B
12 False
13 >>> A == C
14 True
```

